Université de Béjaïa

Cycle ingénieur — 4e année Informatique

Cahier des charges

VMPGC

Compilateur Toy et machine virtuelle
à ramasse-miettes précis

Projet 4 : machine virtuelle avec ramasse-miettes (*Garbage Collector*) précis

Réalisé par : Mansour Abderezak
Badreddine Mamouni

Encadrant :

Dépôt : https://github.com/bizkda/VMPGC

Version 1.0 — 27 octobre 2026
Année universitaire 2026–2027

Table des matières

1 Contexte et état actuel **2**

2 Objectifs et indicateurs **2**

3 Périmètre du projet **2**

4 Parcours utilisateurs et exigences fonctionnelles **3**

4.1 Personas . . . 3

4.2 User stories . . . 3

4.3 Le langage Toy (résumé) . . . 4

5 Spécifications et contraintes techniques **4**

5.1 Architecture cible . . . 4

5.2 Modèle mémoire . . . 4

5.3 Bytecode . . . 5

5.4 Exigence centrale : la précision . . . 5

5.5 Contraintes de qualité . . . 5

6 Planning, méthodologie et assurance qualité **5**

6.1 Plan d'assurance qualité . . . 6

7 Budget, risques et clauses **6**

7.1 Clauses . . . 7

A Grammaire de Toy **7**

B Programme d'exemple (`tests/list.toy`) **7**

C Glossaire **8**

1 Contexte et état actuel

Le sujet du projet 4 demande d'écrire un compilateur dont le code généré interagit avec un gestionnaire de mémoire dynamique développé par les étudiants. Les fonctionnalités avancées attendues sont un ramasse-miettes de type *Mark-and-Sweep* ou *Stop-and-Copy*, ce qui impose au compilateur de produire des cartes de pile (*stack maps*) pour identifier les pointeurs actifs.

Pourquoi un langage dédié ? Les langages existants (C, Java, Python) sont soit trop vastes à analyser, soit trop permissifs sur les types : en C, un entier peut contenir une adresse, ce qui rend un ramasse-miettes précis impossible. Un petit langage statiquement typé, appelé **Toy**, permet au compilateur de savoir à tout moment quelles variables et quels champs sont des pointeurs.

État actuel (27 octobre 2026) :

— dépôt GitHub `bizkda/VMPGC` créé, avec un `README` et un `.gitignore` ;

— lexer en cours d'écriture, classes de l'AST définies ;

— parser, vérificateur de types, générateur de code et VM : à faire.

2 Objectifs et indicateurs

Objectif stratégique : livrer, à la date de rendu fixée avec l'encadrant, une chaîne complète *source Toy* → *bytecode* → *VM en C* dont le ramasse-miettes précis récupère toute la mémoire inutile sans jamais libérer un objet vivant, et le démontrer par des tests de stress et des mesures.

Indicateur	Critère de succès
Programmes de test	100 % des fichiers `.toy` de `tests/` donnent la sortie attendue
Mémoire bornée	`list.toy` (1 000 000 d'allocations) s'exécute avec un tas de taille fixe (valeur à fixer, par exemple 8 Mo) sans dépassement
Sûreté du GC	en mode stress (collecte à chaque allocation) : 0 erreur du vérificateur de tas
Structures cycliques	listes et graphes circulaires correctement récupérés
Comparaison	rapport de mesures (temps, nombre de collectes, octets récupérés) pour chaque GC implémenté
Qualité du code C	0 erreur sous AddressSanitizer et UBSan sur toute la suite de tests

3 Périmètre du projet

Inclus (*In*)	Exclus (*Out*)
Lexer, parser, vérificateur de types (Python)	Génération de code natif x86-64 ou ARM
Générateur de bytecode avec descripteurs de types et stack maps	Ramasse-miettes générationnel, incrémental ou concurrent
VM en C : chargeur, interpréteur, tas	Multithreading

Inclus (*In*)	**Exclus (*Out*)**
Mark-and-Sweep précis	Chaînes de caractères, flottants, bibliothèque standard
Stop-and-Copy (algorithme de Cheney)	Héritage, méthodes, génériques, tableaux multidimensionnels
Mode stress, vérificateur de tas, statistiques de GC	Optimisations du compilateur (SSA, propagation de constantes)
Tests, benchmarks, documentation	Interface graphique, débogueur

4 Parcours utilisateurs et exigences fonctionnelles

4.1 Personas

— **Développeur Toy** : écrit des programmes `.toy`, les compile et les exécute.

— **Évaluateur (encadrant)** : compile, exécute, active le mode stress et lit les statistiques.

— **Auteurs** : font évoluer le compilateur et la VM.

4.2 User stories

ID	**En tant que… je veux… afin de…**	**Critères d'acceptation**
US1	En tant que développeur Toy, je veux déclarer des structs, des tableaux et des fonctions afin de construire des listes et des arbres	`list.toy` et `arrays.toy` compilent
US2	En tant que développeur Toy, je veux des erreurs de syntaxe et de types précises afin de corriger mon code	message au format `ligne:colonne: description` ; erreurs testées (affectation d'un `int` à une struct, mauvais arguments d'appel, champ inexistant)
US3	En tant que développeur Toy, je veux voir le résultat de `print` afin de vérifier mon programme	la sortie est identique à la sortie attendue du test
US4	En tant que développeur Toy, je veux ne jamais libérer la mémoire moi-même afin de me concentrer sur la logique	un programme qui crée des millions d'objets temporaires ne dépasse pas la taille du tas
US5	En tant qu'évaluateur, je veux choisir le GC et la taille du tas en ligne de commande afin de les comparer	options `-gc=ms` ou `-gc=copy`, `-heap=<octets>` et `-gc-stats`
US6	En tant qu'évaluateur, je veux un mode stress et un vérificateur de tas afin de contrôler la précision du GC	`-stress` lance une collecte à chaque allocation ; après chaque collecte, le vérificateur contrôle que tout pointeur atteignable désigne un objet valide

ID	En tant que... je veux... afin de...	Critères d'acceptation
US7	En tant que développeur Toy, je veux être protégé des erreurs d'exécution afin de comprendre ce qui se passe	accès hors bornes, déréférencement de `null` et division par zéro arrêtent le programme avec un message clair, sans plantage du système

4.3 Le langage Toy (résumé)

— Types : `int`, `bool`, types struct (références), tableaux `T[]` et valeur `null`.

— Instructions : `var`, affectation, `if` / `else`, `while`, `return`, `print(expr);` et appel de fonction.

— Expressions : opérateurs arithmétiques, de comparaison et logiques, `new T`, `new T[n]`, accès `a.champ`, indexation `a[i]` et appels.

— Point d'entrée : `fun main(): int`.

5 Spécifications et contraintes techniques

5.1 Architecture cible

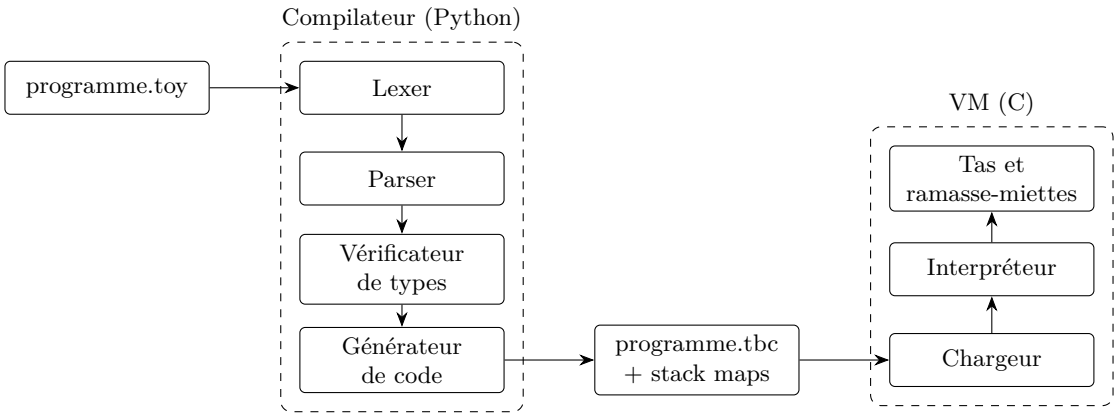


FIGURE 1 – Chaîne complète : du programme Toy à l'exécution

Élément	Choix
Compilateur	Python 3.10 ou plus (dataclasses avec `kw_only`)
VM	C11, gcc ou clang, Makefile
Tests	scripts Python et programmes `.toy` avec sortie attendue
Versionnement	Git, dépôt `github.com/bizkda/VMPGC`, branche `main`

5.2 Modèle mémoire

— Toutes les valeurs font 64 bits ; `null` vaut 0.

— Chaque objet commence par un en-tête contenant le `type_id`, un bit de marque (Mark-and-Sweep) ou un pointeur de redirection (Stop-and-Copy), et la taille ou la longueur du tableau. La taille exacte de l'en-tête (8 ou 16 octets) sera fixée à la conception, au jalon M4.

— Le compilateur émet une table de descripteurs de types : pour chaque `type_id`, les décalages des champs qui sont des pointeurs. Un tableau d'`int` et un tableau de références ont des `type_id` différents.

5.3 Bytecode

Machine à pile : `PUSH_INT`, `LOAD_LOCAL`, `STORE_LOCAL`, opérations arithmétiques et comparaisons, `JMP`, `JZ`, `CALL`, `RET`, `NEW`, `NEW_ARRAY`, `GETFIELD`, `PUTFIELD`, `ALOAD`, `ASTORE`, `PRINT`. La liste définitive sera arrêtée pendant M3.

5.4 Exigence centrale : la précision

— Le compilateur simule les types de la pile d'opérandes pendant la génération du code.

— À chaque point de sûreté (`NEW`, `NEW_ARRAY`, `CALL`), il émet une stack map : un bitmap des variables locales et des emplacements de la pile d'opérandes qui contiennent des références.

— Le GC parcourt les cadres d'appel, retrouve chaque stack map à partir de l'adresse de retour, et ne traite comme racines que les emplacements marqués. Aucun balayage conservatif n'est autorisé.

5.5 Contraintes de qualité

— **Performance :** le temps d'exécution de `list.toy` est mesuré à M4 (sans GC) puis comparé aux deux GC ; les seuils seront fixés après ces mesures.

— **Robustesse :** compilation C avec `-Wall -Wextra`, tests sous `-fsanitize=address, undefined`.

— **Lisibilité :** noms explicites pour les fonctions et les variables (convention du projet).

6 Planning, méthodologie et assurance qualité

Méthode : développement itératif par jalons courts. Chaque jalon se termine par des tests qui passent et un commit. On ne commence pas le jalon suivant tant que le précédent n'est pas validé.

Jalon	Contenu	Critère de sortie	Statut
M1	Lexer, AST, parser	tous les `.toy` d'exemple sont analysés ; les erreurs de syntaxe sont localisées	En cours (lexer, AST)
M2	Vérificateur de types	programmes mal typés rejetés ; chaque expression annotée « pointeur ou non »	À faire
M3	Générateur de code	fichier `.tbc` avec descripteurs de types et stack maps ; désassembleur lisible	À faire
M4	VM sans GC	tas simple avec `malloc` ; les tests de sortie passent	À faire

Jalon	Contenu	Critère de sortie	Statut
M5	Mark-and-Sweep précis	`list.toy` tient dans un tas borné	À faire
M6	Stop-and-Copy (Cheney)	mêmes tests et mêmes sorties qu'avec M5	À faire
M7	Mode stress, vérificateur de tas, benchmarks	0 erreur en stress ; rapport de mesures	À faire

Les dates des jalons sont à fixer avec l'encadrant. Les jalons M1 à M5 sont prioritaires : le sujet demande un GC de type Stop-and-Copy **ou** Mark-and-Sweep, donc M5 couvre déjà l'exigence. M6 et M7 enrichissent le résultat.

6.1 Plan d'assurance qualité

— Tests unitaires du lexer, du parser et du vérificateur de types.

— Tests de bout en bout : chaque `.toy` est accompagné d'un fichier de sortie attendue.

— Tests négatifs : des programmes invalides qui doivent produire une erreur précise.

— Mode stress et vérificateur de tas dès M5.

— Un commit par étape avec un message clair ; aucun jalon n'est validé sans tests.

7 Budget, risques et clauses

Budget : projet académique sans coût financier ; la ressource est le temps. *Build* = développement des jalons M1 à M7 ; *Run* = démonstration, documentation et corrections. Il est recommandé de garder environ 20 % du temps en réserve pour les aléas, surtout pour M3 et M5, les deux jalons les plus délicats.

Risque	Impact	Parade
Stack map incorrecte : un objet vivant est libéré	Élevé	mode stress, vérificateur de tas, tests sur listes, arbres et cycles
Pointeurs non mis à jour après une copie (Cheney)	Élevé	vérificateur de tas après chaque collecte ; comparaison des sorties avec Mark-and-Sweep
Bugs mémoire dans la VM en C	Élevé	AddressSanitizer, UBSan, valgrind
Vérificateur de types plus long que prévu	Moyen	réduire le langage (retirer `bool` ou les tableaux) et noter la décision
Dérive de périmètre	Moyen	section « Exclus » ; tout ajout passe par une mise à jour de ce document
Manque de temps	Moyen	M1 à M5 d'abord, puis M6 ; M7 d'abord en version minimale (mode stress)

7.1 Clauses

— **Licence :** à choisir avant publication (par exemple MIT).

— **Propriété et responsabilité :** le code appartient aux auteurs, qui restent responsables de ce qu'ils rendent et doivent pouvoir expliquer chaque partie.

— **Niveaux de service (SLA/SLO) :** sans objet pour un projet académique.

— **Réversibilité :** le dépôt Git contient le code, les tests et la documentation nécessaires pour qu'une autre personne reprenne le projet.

A Grammaire de Toy

```
program     = { struct_decl | fun_decl } ;
struct_decl = 'struct' ID '{' { ID ':' type ';' } '}' ;
fun_decl    = 'fun' ID '(' [ param { ',' param } ] ')' ':' type block ;
param       = ID ':' type ;
type        = ( 'int' | 'bool' | ID ) { '[' ']' } ;
block       = '{' { statement } '}' ;
statement   = 'var' ID ':' type [ '=' expr ] ';'
            | lvalue '=' expr ';'
            | 'if' '(' expr ')' block [ 'else' ( block | statement ) ]
            | 'while' '(' expr ')' block
            | 'return' [ expr ] ';'
            | 'print' '(' expr ')' ';'
            | expr ';'
            | block ;
expr        = unary { binary_op unary } ;
unary       = ( '-' | '!' ) unary | postfix ;
postfix     = primary { '.' ID | '[' expr ']' } ;
primary     = INT | 'true' | 'false' | 'null'
            | ID [ '(' [ args ] ')' ]
            | 'new' base_type [ '[' expr ']' ]
            | '(' expr ')' ;
```

Priorités des opérateurs binaires, de la plus faible à la plus forte : `||`, `&&`, `== !=`, `< <= > >=`, `+ -`, `* / %`. Les commentaires commencent par `//`.

B Programme d'exemple (`tests/list.toy`)

```
struct Node { val: int; next: Node; }

fun push(head: Node, v: int): Node {
    var n: Node = new Node;
    n.val = v;
    n.next = head;
    return n;
}

fun main(): int {
    var list: Node = null;
    var i: int = 0;
    while (i < 1000000) {
        list = push(null, i);   // les anciennes listes deviennent des déchets
        i = i + 1;
    }
    print(list.val);
    return 0;
```

```
}
```

C Glossaire

GC précis
ramasse-miettes qui sait exactement quels emplacements contiennent des pointeurs.

Racine
variable locale ou valeur de pile qui référence un objet du tas ; point de départ du parcours.

Stack map
table qui indique, pour un point du programme, quels emplacements du cadre d'appel contiennent des références.

Point de sûreté (*safepoint*)
instruction où une collecte peut avoir lieu (allocation, appel).

Mark-and-Sweep
marque les objets atteignables depuis les racines, puis libère tous les autres.

Stop-and-Copy (Cheney)
copie les objets vivants d'une moitié du tas vers l'autre, en laissant des pointeurs de redirection.

Descripteur de type
table qui décrit la disposition d'un objet, en particulier quels champs sont des pointeurs.